Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

LENGUAJE DE PROGRAMACIÓN II

Examen Parcial II — Parte práctica

Versión A · Caso: AutoAndes, órdenes de servicio de un taller mecánico

Docente: Benjamin David Reyna Barreto

Dato	Detalle	Dato	Detalle
Asignatura	Lenguaje de Programación II	Ciclo	IV · Semestre 2026-2
Unidades	1. Backend REST empresarial · 2. Frontend SPA empresarial seguro	Sesión	12 (evaluación de la Unidad II)
Modalidad	Individual: producto propio y defensa individual	Duración	180 minutos continuos
Tecnologías	Spring Boot 4 · Oracle · Angular 22	Fecha	29 de octubre de 2026
Nivel	Intermedio · Grupo A	Puntaje	20 puntos (5 preguntas de 4 puntos)
Docente	Benjamin David Reyna Barreto	Evaluación	Examen Parcial II · Parte práctica

Apellidos y nombres	Código	Grupo	Fecha	Nota
			29 de octubre de 2026	

1. Indicaciones generales

Este examen evalúa, en un solo producto, lo aprendido en las dos primeras unidades: el backend REST empresarial (Unidad 1) y el frontend SPA empresarial seguro (Unidad 2). En 180 minutos construirás, para el caso AutoAndes, un servicio REST con Spring Boot protegido con JWT y una SPA en Angular que lo consume. El examen tiene **cinco preguntas de 4 puntos**; cada pregunta es una parte del producto y se califica por observación directa en ejecución y por revisión del código.

Pregunta	Unidad y sesiones	Qué se evalúa	Puntaje
1. Backend base y CRUD	Unidad 1 · Sesiones 1 a 3	Proyecto, conexión ORM, entidades relacionadas, DTO, validaciones, errores, logs, OpenAPI y CRUD de Servicios y Vehículos.	4 pts
2. Orden de servicio y reporte	Unidad 1 · Sesiones 4 y 5	Registro cabecera–detalle atómico, anulación, consulta con filtros, reporte agregado y CORS.	4 pts
3. Seguridad backend	Unidad 2 · Sesión 10	Usuarios, BCrypt, login JWT, roles y permisos, protección de endpoints y respuestas 401/403.	4 pts
4. SPA y CRUD	Unidad 2 · Sesiones 7 y 8	Proyecto Angular, layout, navegación con carga diferida, CRUD de Servicios y de Vehículos con su lista desplegable.	4 pts
5. Orden de servicio en la SPA y seguridad frontend	Unidad 2 · Sesiones 9 y 11	Formulario cabecera–detalle, consulta, login, sesión, interceptores, guards y menú por permisos.	4 pts

1.1 Distribución del tiempo

El examen se desarrolla en una única jornada de 180 minutos continuos. Al cierre de cada bloque se evalúa lo que exista en tus repositorios en ese momento.

Bloque	Duración	Actividad
1	10 min	Indicaciones, creación de los dos repositorios, verificación de Oracle, del JDK, de Node y del CLI de Angular.
2	75 min	Backend: preguntas 1, 2 y 3. Tiempo sugerido: 30, 25 y 20 minutos. Prueba cada endpoint en Swagger o Postman antes de pasar a la siguiente.
3	75 min	Frontend: preguntas 4 y 5. Tiempo sugerido: 35 y 40 minutos. El docente recorre los puestos y verifica en ejecución lo terminado.
4	20 min	Demostración final y defensa: el docente elige dos preguntas del banco de la sección 7; subida de ramas y etiqueta del commit evaluado.

1.2 Condiciones

- El examen es individual en todas sus partes. No se permite copiar código de otro estudiante ni recibir ayuda; dos entregas con el mismo código se califican ambas con cero.
- Puedes usar como plantilla tus propios repositorios PharmaBackend y PharmaSoft (configuración, clases de seguridad, layout), pero el dominio, las rutas y las reglas deben ser los de este caso. Un archivo que conserve nombres de PharmaSoft (producto, venta, categoría) se califica como no logrado.
- Se permite consultar la documentación oficial de Spring, Angular y Oracle. No se permite el uso de redes sociales, mensajería ni repositorios ajenos.
- El uso de asistentes de inteligencia artificial solo está permitido si explicas y justificas cada línea durante la defensa. Si no puedes explicar el código de una pregunta, esa pregunta se califica con cero.
- Crea los repositorios autoandes-backend y autoandes-frontend, cada uno con las ramas main y develop, y trabaja en la rama examen-a-<apellido>. Registra al menos un commit por pregunta (mínimo seis en total, en los dos repositorios), con la convención de la sección 6.

- El backend debe levantarse con un solo comando y la SPA con ng serve. Lo que solo exista en capturas no se califica.

Prohibición explícita

No se admiten datos escritos a mano en la SPA ni servicios simulados: toda lista proviene del backend. Está prohibido autorizar las operaciones comparando el nombre del rol dentro de un controlador, calcular el total en la SPA y guardarlo tal como llega, y resolver el reporte con findAll() y bucles en Java.

Tampoco se acepta una seguridad que solo viva en la pantalla: una acción oculta en el menú debe ser rechazada igualmente por el backend con 403.

2. Presentación del caso

2.1 La organización

AutoAndes es un taller mecánico con dos locales que atiende a unos seiscientos clientes particulares y pequeñas flotas de taxis. Ofrece servicios como cambio de aceite, alineamiento, afinamiento y revisión de frenos.

Hoy las órdenes de servicio se llenan en talonarios de papel: el total se suma a mano, se repiten servicios por error y nadie sabe cuánto ingresó por cada servicio en el mes. Además, a veces se registra un kilometraje menor que el de la visita anterior, lo que invalida las garantías.

2.2 El alcance del proyecto

La gerencia aprobó construir AutoAndes API, un backend REST con Spring Boot sobre Oracle, y AutoAndes Web, una SPA en Angular que usarán los asesores en el mostrador y el jefe de taller en su oficina.

Las computadoras del mostrador se comparten entre turnos y no todos pueden hacer lo mismo: el asesor registra vehículos y órdenes, pero solo el jefe de taller anula órdenes, mantiene el catálogo de servicios y ve los reportes.

El criterio central de este examen

No se evalúa únicamente que la pantalla muestre datos. Se evalúa que las reglas y la seguridad vivan en el backend, que una orden se guarde completa o no se guarde, y que la SPA ofrezca a cada usuario solo lo que le corresponde sin depender de ello para protegerse.

2.3 Nivel y andamiaje entregado

Esta versión está calibrada para un nivel intermedio. Para que el tiempo alcance, se te entrega resuelto el análisis; a cambio, se exige rigor en dónde ubicas cada responsabilidad.

- El modelo de datos y el script de creación con los datos semilla se entregan en las secciones 3.1 y 9: debes usarlos tal cual, como migración de Flyway.
- La matriz de roles y permisos de la sección 3.2 está resuelta: tu tarea es implementarla, no diseñarla.

- El contrato del API de la sección 3.4 es obligatorio: respeta rutas, verbos, nombres de campo y códigos de estado.
- Una regla de negocio verificada solo en la SPA, o un permiso verificado solo en el menú, se califica como no logrado aunque la pantalla se vea correcta.

3. Modelo, permisos, reglas y contrato

3.1 Modelo de datos

Tabla	Tipo	Campos principales
cliente	Solo lectura (semilla)	id, documento (único), nombres, telefono
servicio	Tabla independiente	id, codigo (único, SRV-000), nombre, precio, duracion_min, activo
vehiculo	Tabla dependiente	id, placa (única, ABC-123), marca, modelo, anio, cliente_id
orden_servicio	Cabecera	id, numero (único), fecha, vehiculo*id, kilometraje, estado, total, usuario*registro, motivo_anulacion
detalle_orden	Detalle	id, orden*id, servicio*id, cantidad, precio_unitario, subtotal
usuarios · roles · usuario_roles	Seguridad	usuarios (id, username, password, nombre*completo, activo); roles (id, nombre); usuario*roles (usuario*id, rol*id), muchos a muchos

Relaciones: un cliente tiene muchos vehículos; un vehículo tiene muchas órdenes; una orden tiene muchos detalles y cada detalle referencia un servicio; un usuario tiene uno o más roles.

3.2 Roles y permisos

Los permisos se conceden por rol y se verifican siempre contra el permiso, nunca contra el nombre del rol. Un usuario con varios roles recibe la unión de sus permisos.

Rol	Quién es	Qué hace en el sistema
ADMIN	Administrador del sistema	Tiene todos los permisos.
ASESOR	Asesor de servicio en el mostrador	Registra vehículos y órdenes, y consulta servicios y órdenes.
JEFE_TALLER	Jefe de taller	Mantiene el catálogo de servicios, anula órdenes y consulta reportes; no registra órdenes.

Permiso	Permite	ADMIN	ASESOR	JEFE_TALLER
SERVICIO_LEER	Consultar servicios	Sí	Sí	Sí
SERVICIO_ESCRIBIR	Registrar, editar y dar de baja servicios	Sí	No	Sí
VEHICULO_LEER	Consultar vehículos y clientes	Sí	Sí	Sí
VEHICULO_ESCRIBIR	Registrar, editar y eliminar vehículos	Sí	Sí	No

Permiso	Permite	ADMIN	ASESOR	JEFE_TALLER
ORDEN_LEER	Consultar órdenes y su detalle	Sí	Sí	Sí
ORDEN_REGISTRAR	Registrar órdenes	Sí	Sí	No
ORDEN_ANULAR	Anular órdenes	Sí	No	Sí
REPORTE_VER	Ver el reporte de ingresos	Sí	No	Sí

Dónde se verifica el permiso

La verificación obligatoria ocurre en el backend: una petición sin el permiso necesario responde 403 aunque llegue con un token válido. En la SPA, ocultar el menú o el botón es una mejora de experiencia, no un control de seguridad. En la defensa se te pedirá invocar el endpoint con el token de un usuario sin el permiso y mostrar el rechazo.

3.3 Reglas de negocio

Las reglas se implementan en la capa de servicio del backend, no en el controlador ni en la SPA. La SPA debe mostrar el mensaje que devuelve el backend.

Código	Regla
RN-01	El código de servicio es único (409). Un servicio usado en alguna orden no se elimina físicamente: DELETE responde 409 y se ofrece PATCH /servicios/{id}/estado para darlo de baja.
RN-02	La placa es única (409) y el cliente debe existir (404). Un vehículo con órdenes registradas no se puede eliminar (409).
RN-03	Una orden tiene entre 1 y 10 detalles, sin repetir el mismo servicio, con cantidad de 1 a 5 (400). Todos los servicios deben estar activos (409).
RN-04	El kilometraje no puede ser menor que el de la última orden REGISTRADA del mismo vehículo (409, con el último kilometraje en el mensaje).
RN-05	Solo se anula una orden REGISTRADA y en el mismo día de su registro; el motivo es obligatorio, de 10 a 200 caracteres (400 o 409 según el caso).

3.4 Contrato del API

Todas las rutas van bajo /api/v1. Salvo el login, todas exigen el token.

Verbo	Ruta	Permiso	Códigos
POST	/auth/login	Público	200 · 400 · 401
GET	/auth/me	Autenticado	200 · 401
GET	/clientes	VEHICULO_LEER	200 · 401 · 403
GET · POST	/servicios	SERVICIO*LEER* · *SERVICIO*ESCRIBIR	200 · 201 · 400 · 401 · 403 · 409

Verbo	Ruta	Permiso	Códigos
GET · PUT · DELETE	/servicios/{id}	SERVICIO*LEER* · *SERVICIO*ESCRIBIR	200 · 204 · 400 · 404 · 409
PATCH	/servicios/{id}/estado	SERVICIO_ESCRIBIR	200 · 401 · 403 · 404
GET · POST	/vehiculos?clienteId	VEHICULO*LEER* · *VEHICULO*ESCRIBIR	200 · 201 · 400 · 404 · 409
GET · PUT · DELETE	/vehiculos/{id}	VEHICULO*LEER* · *VEHICULO*ESCRIBIR	200 · 204 · 400 · 404 · 409
POST	/ordenes	ORDEN_REGISTRAR	201 · 400 · 401 · 403 · 404 · 409
GET	/ordenes?placa&estado&desde&hasta	ORDEN_LEER	200 · 401 · 403
GET	/ordenes/{id}	ORDEN_LEER	200 · 401 · 403 · 404
PATCH	/ordenes/{id}/anular	ORDEN_ANULAR	200 · 400 · 403 · 404 · 409
GET	/reportes/ingresos-por-servicio?desde&hasta	REPORTE_VER	200 · 401 · 403

3.5 Formato de error común

```
{
  "timestamp": "2026-10-29T09:14:22",
  "status": 400,
  "error": "Bad Request",
  "mensaje": "Datos inválidos",
  "ruta": "/api/v1/vehiculos",
  "campos": { "placa": "La placa debe tener el formato ABC-123" }
}
```

Todas las respuestas de error (400, 401, 403, 404, 409 y 500) usan este formato; «campos» solo aparece en los errores de validación.

4. Preguntas del examen

Desarrolla las cinco preguntas en orden. Cada una se apoya en la anterior, pero se califica por separado: una pregunta incompleta no anula las siguientes si estas pueden demostrarse.

Pregunta 1 · Backend base y CRUD de tablas independiente y dependiente (4 puntos)

Unidad 1 · Sesiones 1 a 3 · Arquitectura REST, CRUD completo, objetos relacionados

Crea el proyecto autoandes-backend con Spring Boot, conéctalo a Oracle, aplica el script de la sección 9 como migración de Flyway e implementa el CRUD de Servicios y de Vehículos, además de la consulta de clientes.

Requerimiento	Comportamiento esperado
Arquitectura en capas	Paquetes config, entity, repository, service, controller, dto, mapper y exception. El controlador nunca usa el repositorio directamente.
Entidades y relación	Entidades JPA de las tablas de la sección 3.1. Vehiculo se relaciona con Cliente mediante @ManyToOne con carga diferida.
DTO	Request y Response separados de las entidades. VehiculoResponseDTO incluye clienteId y clienteNombre, nunca la entidad anidada.
Validaciones	Bean Validation en los Request: código obligatorio con el patrón SRV-000, nombre de 3 a 80 caracteres, precio mayor que cero y duración entre 15 y 480 minutos; placa con el patrón ABC-123, marca y modelo obligatorios, año entre 1990 y 2027 y clienteId obligatorio.
Errores	GlobalExceptionHandler con el formato de la sección 3.5: 400 validación, 404 recurso inexistente, 409 regla de negocio (RN-01 y RN-02), 500 inesperado.
Logs y documentación	Log INFO en cada alta, cambio y baja, y WARN en cada regla rechazada. Swagger UI disponible con todos los endpoints.

Evidencia en ejecución: desde Swagger, registra, edita, consulta y da de baja un registro de cada recurso, y provoca un 400, un 404 y un 409.

Cómo se califica esta pregunta (4 puntos)

Logrado, 4 puntos: Los dos CRUD funcionan contra Oracle, con DTO, relación correcta, validaciones y los códigos 400, 404 y 409 con el formato común.

En proceso, 2,4 puntos: Funcionan los CRUD, pero exponen entidades, el error no sigue el formato común o falta alguna regla.

Inicial, 1,2 puntos: Solo funcionan algunas consultas o el proyecto no conecta con la base de datos.

No presentado o no demostrado en ejecución: 0 puntos.

Pregunta 2 · Orden de servicio cabecera–detalle, consulta y reporte (4 puntos)

Unidad 1 · Sesiones 4 y 5 · Operación transaccional, consultas empresariales, reportes y CORS

Implementa el registro de órdenes de servicio con su detalle, su anulación, la consulta con filtros y el reporte de ingresos por servicio, y habilita CORS para la SPA.

Requerimiento	Comportamiento esperado
Registro atómico	POST /ordenes recibe vehiculoId, kilometraje y la lista de detalles (servicioId, cantidad). Guarda cabecera y detalles en una sola transacción (@Transactional en el servicio) y responde 201 con la orden completa.
Cálculos y estado	El precio unitario se toma de la base de datos; subtotal = precio × cantidad y total = suma de subtotales, calculados en el servicio. La orden nace REGISTRADA con número OS-000001 correlativo.
Reglas	RN-03 y RN-04: cualquier detalle inválido revierte toda la operación (rollback) y responde con su código.
Anulación	PATCH /ordenes/{id}/anular con motivo, según RN-05; responde 200 con la orden ANULADA.
Consulta	GET /ordenes?placa&estado&desde&hasta con filtros combinables y opcionales, ordenada por fecha descendente; sin resultados responde 200 con lista vacía.
Reporte	GET /reportes/ingresos-por-servicio?desde&hasta con una consulta JPQL agregada (GROUP BY) y una proyección: código, servicio, cantidad total e importe total, ordenado por importe descendente, excluyendo las órdenes ANULADAS.
CORS	Origen http://localhost:4200, métodos GET, POST, PUT, PATCH, DELETE y OPTIONS, y el encabezado Authorization.

Evidencia en ejecución: registra una orden correcta y muestra en SQL Developer la cabecera, los detalles y el total; luego provoca el rechazo de una orden con un servicio inactivo en el tercer detalle y demuestra que no quedó nada guardado a medias.

Cómo se califica esta pregunta (4 puntos)

Logrado, 4 puntos: Registro atómico con cálculos en el servicio, reglas con su código, anulación, consulta con filtros combinados, reporte agregado con JPQL y CORS configurado.

En proceso, 2,4 puntos: El registro funciona, pero sin rollback demostrable, con cálculos que vienen del cliente o con el reporte resuelto en memoria.

Inicial, 1,2 puntos: Se guarda la cabecera sin detalles o los detalles quedan huérfanos ante un error.

No presentado o no demostrado en ejecución: 0 puntos.

Pregunta 3 · Seguridad backend con JWT, roles y permisos (4 puntos)

Unidad 2 · Sesión 10 · Usuarios, hash de contraseñas, autenticación JWT, roles, permisos y protección de endpoints

Protege AutoAndes API con Spring Security. Las tablas de usuarios, roles y la relación muchos a muchos usuario_roles ya están en el script de la sección 9.

Requerimiento	Comportamiento esperado
Contraseñas	Guardadas con BCrypt; el login compara con passwordEncoder.matches y nunca devuelve la contraseña.

Requerimiento	Comportamiento esperado
Login JWT	POST /auth/login devuelve token, tipo Bearer, expiraEnSegundos (3600), nombre completo, roles y permisos. Mismo mensaje 401 para usuario inexistente, contraseña incorrecta y cuenta inactiva.
Filtro y sesión	API sin estado (STATELESS); un filtro valida firma y expiración del token en cada petición. Solo /auth/login, Swagger y OPTIONS son públicos.
Autorización	Permisos por rol según la matriz 3.2, declarados con hasAuthority en la configuración o con @PreAuthorize (por ejemplo, en el reporte). Ningún controlador compara nombres de rol.
401 y 403	Sin token, token alterado o vencido: 401 con el formato común. Autenticado sin permiso: 403 con el formato común.
Usuario actual	GET /auth/me devuelve los datos y permisos del usuario del token.

Evidencia en ejecución: login de los tres usuarios; el ASESOR intenta anular una orden y ver el reporte (403), y el JEFE_TALLER intenta registrar una orden (403); una petición sin token y otra con el token alterado.

Cómo se califica esta pregunta (4 puntos)

Logrado, 4 puntos: BCrypt, token firmado con expiración, filtro que protege todas las rutas, permisos centralizados y distinción correcta entre 401 y 403.

En proceso, 2,4 puntos: La autenticación funciona, pero los permisos se revisan por nombre de rol, alguna ruta queda abierta o 401 y 403 se confunden.

Inicial, 1,2 puntos: Hay login, pero el token no se valida o todas las rutas quedan abiertas.

No presentado o no demostrado en ejecución: 0 puntos.

Pregunta 4 · SPA en Angular: layout, navegación y CRUD (4 puntos)

Unidad 2 · Sesiones 7 y 8 · Arquitectura de la SPA, CRUD de tablas independientes y dependientes

Crea el proyecto autoandes-frontend con Angular y componentes standalone, y construye los módulos de Servicios y Vehículos consumiendo tu backend.

Requerimiento	Comportamiento esperado
Estructura	Carpetas core (servicios, modelos, guards, interceptores, configuración), shared, layout y features. URL de la API en environment.
Layout y navegación	Encabezado, menú lateral y router-outlet; rutas hijas con loadChildren o loadComponent; routerLinkActive en el menú; página 404.
CRUD de Servicios	Lista con buscador, formulario reactivo único para registrar y editar (/servicios/nuevo y /servicios/:id/editar), confirmación antes de dar de baja y estados de carga, vacío y error.
CRUD de Vehículos	Lista que muestra la placa, la marca, el modelo, el año y el nombre del cliente; formulario con lista desplegable de clientes cargada desde el backend, validaciones equivalentes a las del backend (placa ABC-123, año entre 1990 y 2027) y filtro por cliente.

Requerimiento	Comportamiento esperado
Errores del backend	Los mensajes de «campos» de un 400 se muestran bajo cada control; el mensaje de un 409 o 404 se muestra en la página y la lista no queda bloqueada.

Evidencia en ejecución: registra y edita un servicio y un vehículo; provoca el 409 de una placa repetida y muestra el mensaje en pantalla.

Cómo se califica esta pregunta (4 puntos)

Logrado, 4 puntos: Estructura por capas, layout con carga diferida, los dos CRUD completos con formulario reactivo único, lista desplegable desde el backend y mensajes del backend en pantalla.

En proceso, 2,4 puntos: Los CRUD funcionan, pero con formularios duplicados, datos relacionados mostrados como identificador o errores genéricos.

Inicial, 1,2 puntos: Solo funcionan las listas o la SPA no consume el backend.

No presentado o no demostrado en ejecución: 0 puntos.

Pregunta 5 · Orden de servicio en la SPA y seguridad frontend (4 puntos)

Unidad 2 · Sesiones 9 y 11 · Formularios cabecera–detalle, consultas, login, sesión, interceptores, guards y menú por permisos

Completa la SPA con el registro y la consulta de órdenes de servicio y protégela con el backend seguro de la pregunta 3.

Requerimiento	Comportamiento esperado
Registro de orden de servicio	Formulario con lista desplegable de vehículos (placa y cliente), kilometraje y detalle dinámico: agregar y quitar líneas, servicio sin repetir, cantidad de 1 a 5, subtotal y total recalculados al instante; botón deshabilitado mientras se envía y mensaje con el número de la orden al registrar; el 409 se muestra sin perder las líneas.
Consulta	Lista de órdenes de servicio con los filtros del contrato enviados como parámetros, detalle de una orden y acción «Anular» con motivo.
Login y sesión	Login fuera del layout, sesión en un servicio con signals y en sessionStorage con su vencimiento; cierre de sesión y cierre automático al vencer el token.
Interceptores	Uno agrega Authorization: Bearer solo a la API; otro, ante un 401 (excepto el del login), cierra la sesión y vuelve al login con returnUrl. Un 403 no cierra la sesión.
Guards y menú	authGuard en el layout, invitadoGuard en el login, permisoGuard con el permiso en data de cada ruta; menú y botones según los permisos; página «Acceso denegado».

Evidencia en ejecución: registra una orden como ASESOR; ingresa como JEFE_TALLER y comprueba que «Nueva orden» no aparece y que /ordenes/nueva escrita a mano muestra «Acceso denegado»; altera el token en sessionStorage y comprueba que la SPA vuelve al login.

Cómo se califica esta pregunta (4 puntos)

Logrado, 4 puntos: Formulario cabecera–detalle con cálculos reactivos y mensajes del backend, consulta con filtros, sesión con expiración, dos interceptores, guards por sesión y por permiso, y menú y botones por permisos.

En proceso, 2,4 puntos: Funciona el registro o la seguridad, pero el token se agrega servicio por servicio, el 403 cierra la sesión o las rutas solo se protegen ocultando el menú.

Inicial, 1,2 puntos: El formulario no registra o la SPA no puede iniciar sesión contra el backend.

No presentado o no demostrado en ejecución: 0 puntos.

5. Estructura de paquetes esperada

Backend:

```
src/main/java/pe/edu/upeu/autoandes/
├── config/        SecurityConfig, JwtService, JwtAuthenticationFilter,
│                  CorsConfig, OpenApiConfig
├── security/      Permiso, PermisosPorRol, UsuarioDetailsService,
│                  RestAuthenticationEntryPoint, RestAccessDeniedHandler
├── entity/        Cliente, Servicio, Vehiculo, OrdenServicio, DetalleOrden,
│                  Usuario, Rol
├── repository/    interfaces JpaRepository y consultas JPQL del reporte
├── service/       interfaces e implementaciones con las reglas RN y @Transactional
├── controller/    AuthController y controladores REST bajo /api/v1
├── dto/           Request, Response, proyecciones del reporte y ErrorResponseDTO
├── mapper/        conversiones entidad ↔ DTO
└── exception/     GlobalExceptionHandler y excepciones del dominio
src/main/resources/db/migration/V1__esquema_y_semilla.sql
```

Frontend:

```
src/app/
├── core/       auth/ (modelo y AuthService), guards/, interceptors/,
│               config/menu.ts, services/ (un servicio HTTP por recurso)
├── shared/     directives/si-permiso, pages/acceso-denegado, pages/no-encontrado
├── layout/     main-layout, header, sidebar
└── features/   auth/login,
                servicios, vehiculos, ordenes (registro y consulta), reportes
src/environments/environment.ts      apiUrl: 'http://localhost:8080/api/v1'
```

6. Versionado y trazabilidad con Git

Aunque los repositorios se crean al inicio del examen, el historial debe mostrar cómo creció el trabajo. Ninguna pregunta se desarrolla directamente sobre main.

```
git checkout -b develop
git checkout -b examen-a-<apellido>
git add .
git commit -m "feat(vehiculos): CRUD con relación a cliente y validaciones"
git push -u origin examen-a-<apellido>
# Al terminar: fusionar en develop y en main, y etiquetar
git tag -a v1.0-examen-u2 -m "Examen Parcial II"
git push origin --tags
```

Prefijo	Cuándo se usa	Ejemplo
feat	Se agrega una funcionalidad.	feat(ordenes): registrar orden de servicio con detalle en una transacción
fix	Se corrige un defecto.	fix(auth): no cerrar la sesión ante un 403
refactor	Se reorganiza sin cambiar el comportamiento.	refactor(seguridad): centralizar permisos por rol
test	Se agregan pruebas o colecciones.	test(api): colección de Postman con 400, 401, 403 y 409

Cómo se aplica el criterio de Git

Se exigen al menos seis commits en total, uno o más por pregunta, con mensajes según la convención y la etiqueta v1.0-examen-u2 en ambos repositorios. Un único commit al final, mensajes como «avance» o «subida final», o trabajo directo sobre main descuentan 2 puntos de la nota del examen.

7. Banco de preguntas de defensa

El docente elige dos preguntas por estudiante. Responder exige mostrar el código propio y el sistema en ejecución. Si no logras explicar tu código, la pregunta del examen a la que corresponde se califica con cero.

1. Muéstrame el recorrido de una orden desde el formulario hasta las tablas de Oracle, nombrando cada archivo por el que pasa.
2. ¿Dónde se calcula el total de la orden y por qué no se acepta el que envía la SPA?
3. Provoca un error en el tercer detalle y demuestra que no quedó nada guardado. ¿Qué anotación lo garantiza y en qué capa está?
4. Muéstrame la consulta JPQL del reporte y explica por qué no se resuelve con findAll() y un bucle.
5. ¿Por qué tu DTO de respuesta no devuelve la entidad con su relación anidada? ¿Qué problemas evitas?
6. Decodifica el token del login: ¿qué transporta, quién lo firma y qué ocurre exactamente cuando vence?
7. Muéstrame dónde se verifica el permiso de esta operación en el backend e invócala con un usuario que no lo tiene.
8. Explica la diferencia entre 401 y 403 en tu SPA y muéstrame el código que los distingue.
9. Muéstrame el punto único donde se agrega el encabezado Authorization. ¿Qué pasaría si se hiciera en cada servicio?
10. ¿Por qué guardas la sesión en sessionStorage y no en localStorage en las computadoras compartidas del caso?

11. Escribe a mano en la barra de direcciones una ruta para la que no tienes permiso y explica qué guard actúa.

12. Explica por qué el menú se actualiza solo al cambiar de usuario, sin recargar la página.

8. Rúbrica de evaluación

El examen se califica sobre veinte puntos. Los niveles equivalen al 100 %, al 60 % y al 30 % del puntaje de cada pregunta.

Criterio	Pts.	Logrado (100 %)	En proceso (60 %)	Inicial (30 %)
P1. Backend base y CRUD	4	Capas separadas, DTO, relación @ManyToOne, validaciones, errores con formato común, logs y Swagger; los dos CRUD operan contra Oracle.	CRUD operativos con entidades expuestas, errores sin formato común o alguna regla faltante.	Solo consultas o sin conexión a la base de datos.
P2. Orden de servicio y reporte	4	Registro atómico con cálculos y reglas en el servicio, anulación, filtros combinados, reporte JPQL agregado y CORS.	Sin rollback demostrable, cálculos tomados del cliente o reporte en memoria.	Cabecera sin detalles o datos a medias ante un error.
P3. Seguridad backend	4	BCrypt, JWT firmado con expiración, filtro STATELESS, permisos centralizados, 401 y 403 correctos.	Permisos por nombre de rol, rutas abiertas o 401 y 403 confundidos.	Token sin validar o rutas abiertas.
P4. SPA y CRUD	4	core/shared/layout/features, carga diferida, dos CRUD con formulario único, lista desplegable y errores del backend en pantalla.	Formularios duplicados, relación mostrada como identificador o errores genéricos.	Solo listas o sin consumo del backend.
P5. Orden de servicio en la SPA y seguridad	4	Cabecera–detalle reactivo, consulta con filtros, sesión con expiración, interceptores, guards y menú por permisos.	Token agregado servicio por servicio, 403 que cierra sesión o rutas protegidas solo por el menú.	No registra o no inicia sesión.

Nota mínima aprobatoria

Conforme al sílabo, la nota mínima aprobatoria es trece. Un estudiante que no logre levantar su backend durante la evaluación no puede superar los ocho puntos, porque sin servicio no hay nada que consumir ni que proteger.

9. Entregables y lista de cotejo

- Enlaces a autoandes-backend y autoandes-frontend con la etiqueta v1.0-examen-u2 en el commit evaluado.
- Colección de Postman exportada con un ejemplo de 200/201, 400, 401, 403, 404 y 409.
- Documento PDF Apellido*LP2*ExamenU2.pdf con capturas: login, CRUD, registro cabecera–detalle, reporte, rechazo 403 y página de acceso denegado.
- README.md en cada repositorio con instrucciones de ejecución, usuarios de prueba y contrato del API.

9.1 Lista de cotejo antes de entregar

N.º	Criterio observable	Sí / No
1	El backend levanta con un comando y aplica la migración de Flyway con los datos semilla.	
2	Los CRUD de Servicios y Vehículos responden 201, 200, 400, 404 y 409 según el caso.	
3	Una orden con un detalle inválido no deja nada guardado.	
4	El reporte se resuelve con una consulta agregada y excluye lo anulado.	
5	Sin token, todas las rutas salvo el login responden 401; sin permiso, 403.	
6	Las contraseñas están cifradas con BCrypt en la base de datos.	
7	La SPA tiene layout, menú y rutas con carga diferida.	
8	Los formularios muestran bajo cada campo los mensajes de validación del backend.	
9	El token se agrega en un interceptor y la sesión se guarda con su vencimiento.	
10	Una ruta escrita a mano sin permiso muestra «Acceso denegado» y la sesión continúa.	
11	Hay al menos seis commits con la convención y la etiqueta v1.0-examen-u2.	

10. Anexo. Script de base de datos y datos semilla

Guárdalo como V1_*esquema*y_semilla.sql. Los hash de contraseña se generan con BCrypt al iniciar (por ejemplo, con un CommandLineRunner que cifra las contraseñas de prueba si la columna está vacía) o se insertan ya cifrados.

Por qué estos datos mínimos

Sin un servicio inactivo no se puede demostrar la regla RN-03; sin un vehículo con una orden previa no se puede demostrar el kilometraje de RN-04; sin una orden anulada no se puede comprobar que el reporte la excluye; y sin un usuario inactivo no se prueba el login rechazado.

```
CREATE TABLE cliente (
  id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  documento VARCHAR2(11) NOT NULL UNIQUE, nombres VARCHAR2(120) NOT NULL,
  telefono VARCHAR2(15));
CREATE TABLE servicio (
  id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  codigo VARCHAR2(7) NOT NULL UNIQUE, nombre VARCHAR2(80) NOT NULL,
  precio NUMBER(10,2) NOT NULL CHECK (precio > 0),
  duracion_min NUMBER(4) NOT NULL, activo NUMBER(1) DEFAULT 1 NOT NULL);
CREATE TABLE vehiculo (
  id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  placa VARCHAR2(7) NOT NULL UNIQUE, marca VARCHAR2(40) NOT NULL,
  modelo VARCHAR2(40) NOT NULL, anio NUMBER(4) NOT NULL,
  cliente_id NUMBER NOT NULL REFERENCES cliente(id));
CREATE TABLE orden_servicio (
  id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  numero VARCHAR2(9) NOT NULL UNIQUE, fecha TIMESTAMP NOT NULL,
  vehiculo_id NUMBER NOT NULL REFERENCES vehiculo(id),
  kilometraje NUMBER(7) NOT NULL, estado VARCHAR2(12) NOT NULL,
  total NUMBER(10,2) NOT NULL, usuario_registro VARCHAR2(30) NOT NULL,
  motivo_anulacion VARCHAR2(200));
CREATE TABLE detalle_orden (
  id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  orden_id NUMBER NOT NULL REFERENCES orden_servicio(id),
  servicio_id NUMBER NOT NULL REFERENCES servicio(id),
  cantidad NUMBER(2) NOT NULL, precio_unitario NUMBER(10,2) NOT NULL,
  subtotal NUMBER(10,2) NOT NULL);
-- usuarios, roles y usuario_roles: igual que en la sesión 10

-- Semilla mínima exigida
-- 6 clientes · 8 servicios (SRV-001 a SRV-008, uno inactivo: SRV-008)
-- 8 vehículos (al menos un cliente con 2) · 3 órdenes REGISTRADAS
--   (una de ellas de AAA-111 con kilometraje 45000) y 1 ANULADA
-- roles ADMIN, ASESOR, JEFE_TALLER y los 4 usuarios de prueba
```

Usuarios de prueba

Usuario	Contraseña	Rol	Estado
admin	Auto2026*	ADMIN	Activo
asesor	Auto2026*	ASESOR	Activo
jefe	Auto2026*	JEFE_TALLER	Activo
inactivo	Auto2026*	ASESOR	Inactivo